

Modélisation du jeu Golden Quest

LENING TALAUPA Zidane Steve

16 Novembre 2025

1 Modélisation du plateau de jeu

Hypothèses :

- Puisqu'un joueur ne peut créer que des niveaux et pas des leçons, nous faisons l'hypothèse que le jeu a au moins une leçon. Il ne serait pas possible d'y jouer autrement.
- On considère la carte d'un niveau comme un repère dont l'origine est le point en haut à gauche. Ainsi, chaque case a une position sur la carte qui est représentée par des coordonnées (une abscisse et une ordonnée).
- Le login d'un joueur c'est son email. C'est le seul attribut qui peut être unique à chaque joueur.

Question 4.1

Première version d'un diagramme de classes UML qui fixe les éléments principaux : voir Figure 1

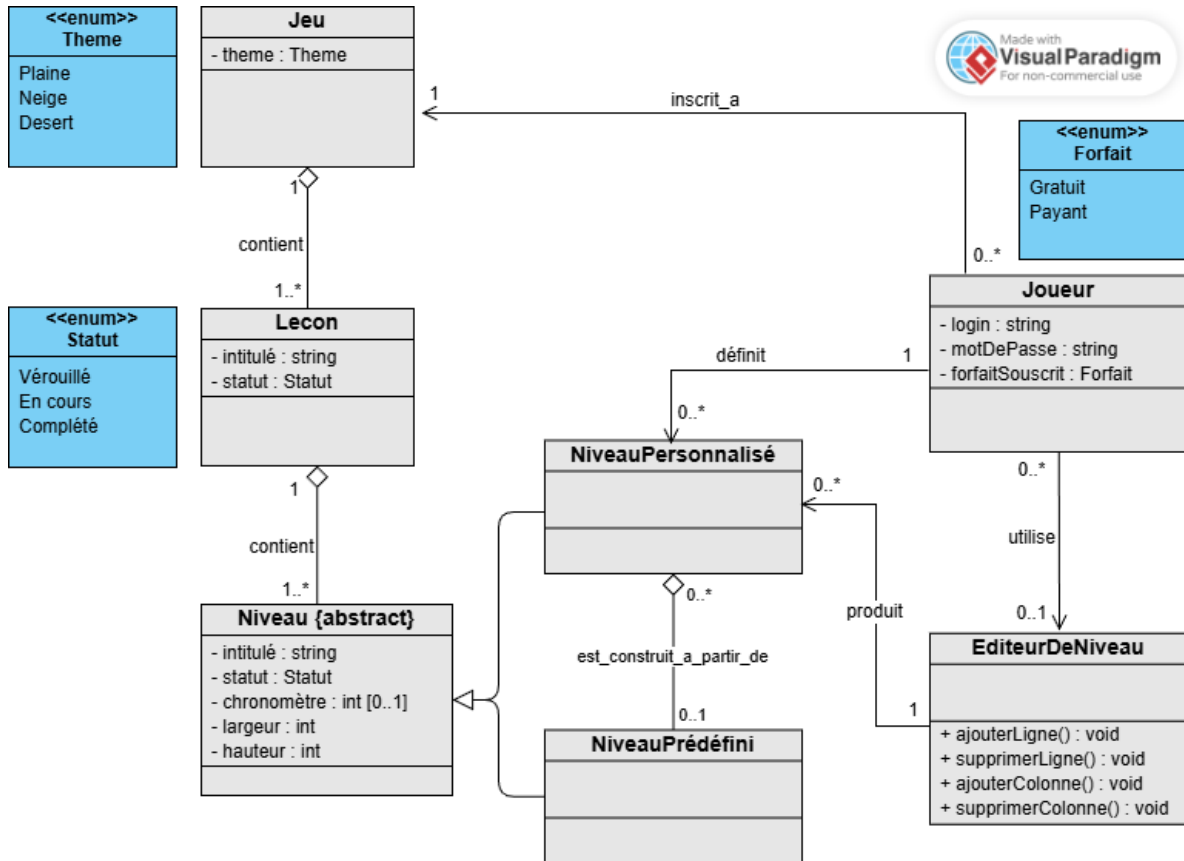


Figure 1: Première version d'un diagramme de classes UML

Question 4.2

Enrichissons cette première version avec les détails sur les joueurs et les niveaux : voir Figure 2

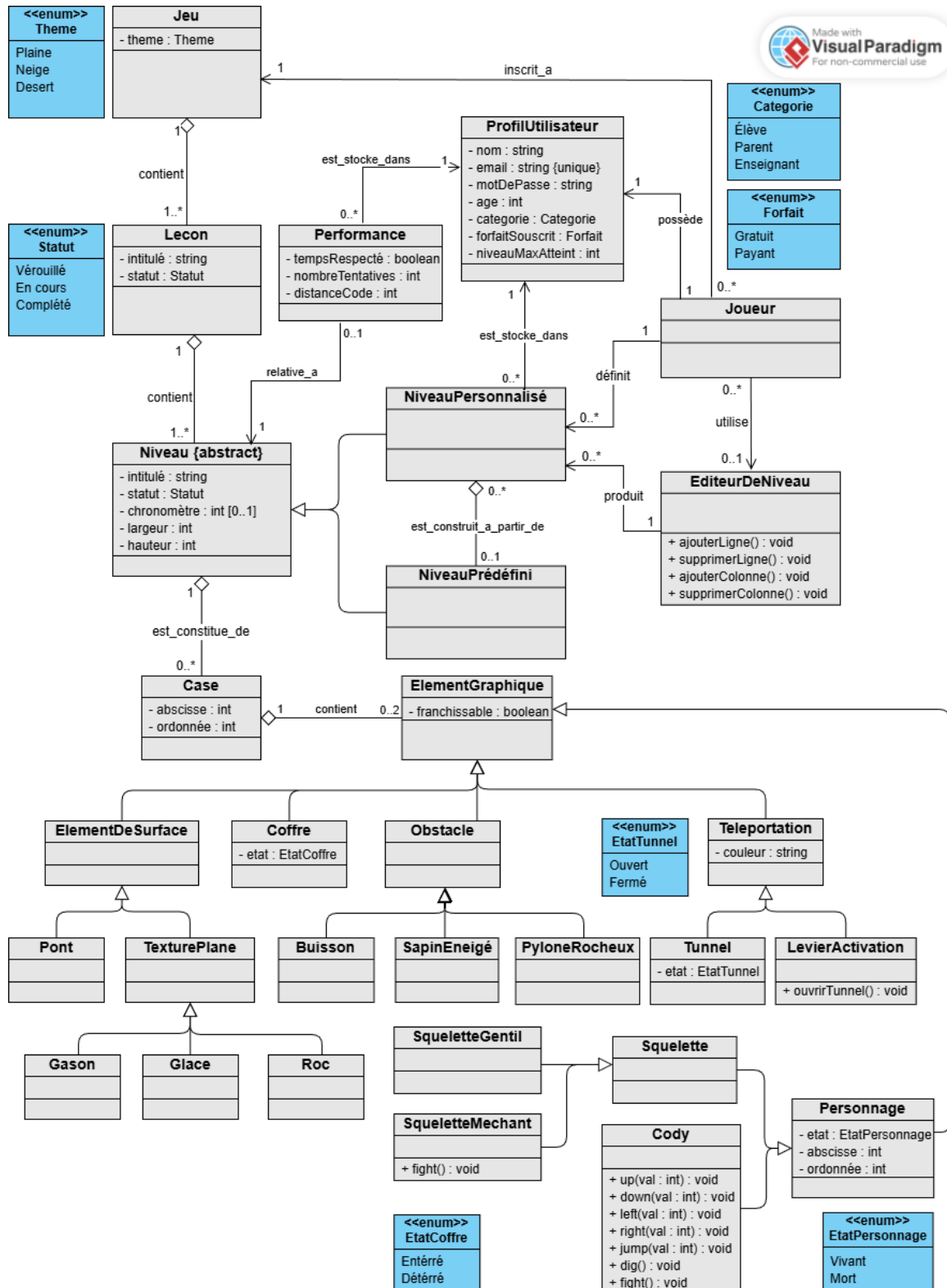


Figure 2: Diagramme de classes enrichi

Question 4.3

Spécifions les contraintes suivantes :

1. Les logins des joueurs inscrits sont tous différents :
Mention {unique} associée à l'attribut email dans ProfilUtilisateur.
2. Une leçon possède au minimum un niveau :
Pris en compte dans le diagramme de classes (cardinalité 1..* de l'agrégation entre Leçon et Niveau)
3. Soit un niveau est prédéfini au sein d'une leçon, soit il est défini par un joueur, mais ne peut pas être les deux en même temps :
Puisque nous avons utilisé l'héritage simple entre Niveau, NiveauPrédéfini et NiveauPersonnalisé, et que Niveau est une classe **abstraite**, alors un niveau est nécessairement soit prédéfini, soit défini par un joueur et jamais les deux en même temps.
4. Les coordonnées d'une case ne peuvent excéder les dimensions du niveau :
On fait le choix de compter les coordonnées à partir de 1. La contrainte OCL correspondante peut s'écrire de la manière suivante :

```
context Case
inv :
(1 <= self.abscisse <= self.niveau.largeur) and
(1 <= self.ordonnee <= self.niveau.hauteur)
```
5. Chaque niveau ne contient qu'un seul Cody, et qu'un seul coffre :
La contrainte OCL correspondante est :

```
context Niveau
inv :
(self.cases.elementGraphiques->select(e | e.oclIsTypeOf(Coffre))->size() = 1)
and
(self.cases.elementGraphiques->select(e | e.oclIsTypeOf(Cody))->size() = 1)
```
6. Un personnage ne peut pas se trouver sur un obstacle (même après un déplacement) :
La contrainte OCL est la suivante :

```
context Personnage
inv :
self.case.elementGraphiques->select(e | e.oclIsTypeOf(Obstacle))->size = 0
```
7. Deux personnages ne peuvent pas partager la même case :
La contrainte OCL est la suivante :

```
context Case
inv :
self.elementGraphiques->select(e | e.oclIsTypeOf(Personnage))->size() <= 1
```
8. Le coffre doit se trouver sur un élément de surface franchissable :
Une case ne peut contenir qu'au plus 2 éléments graphiques. Si la case contient déjà le coffre, il suffit de vérifier que la case contient aussi un élément de surface. La contrainte OCL est la suivante :

```
context Coffre
inv :
self.case.elementGraphiques->select(e | e.oclIsTypeOf(ElementDeSurface))->notEmpty()
```
9. Chaque niveau comporte soit une paire de tunnels de téléportation de même couleur, soit un tunnel unique d'une couleur mais qui est initialement fermé :
On part d'un tunnel, s'il est fermé, on vérifie qu'il est le seul tunnel de cette couleur dans le niveau. S'il est ouvert, on vérifie qu'il existe exactement deux tunnels de cette couleur dans le niveau. Voici la contrainte OCL correspondante :

```
context Tunnel
inv :
```

```

if self.etat = Etat::Fermé
then self.case.niveau.cases.elementGraphiques->select(e | e.ocIsTypeOf(Tunnel) and
e.ocIsType(Tunnel).couleur = self.couleur)->size() = 1
else self.case.niveau.cases.elementGraphiques->select(e | e.ocIsTypeOf(Tunnel) and
e.ocIsType(Tunnel).couleur = self.couleur)->size() = 2
endif

```

10. Un levier de téléportation ne peut être présent que s'il existe des tunnels de la même couleur :

```

context LevierActivation
inv :
self.case.niveau.cases.elementGraphiques->select(e | e.ocIsTypeOf(Tunnel)
and e.ocIsType(Tunnel).couleur = self.couleur)->notEmpty()

```

11. Un obstacle (buisson, sapin ou pylône rocailloux) doit obligatoirement être posé sur une surface franchissable :

En suivant le même raisonnement que celui du coffre, on a la contrainte OCL ci-dessous :

```

context Obstacle
inv :
self.case.elementGraphiques->select(e | e.ocIsTypeOf(ElementDeSurface))->notEmpty()

```

Question 4.4

Pour la Figure 3 fournie,

1. Définissons le niveau décrit à l'aide d'un Diagramme d'Objets UML, en cohérence avec le Diagramme de Classe obtenu au terme de la Question 4.2 : voir Figure 3

Dans un soucis de complétude, j'ai ajouté un **objet hypothétique** représentant le tonneau qui se trouve à la position (2,2) de la carte. Il est évident que la classe Tonneau n'existe pas dans notre diagramme de classes, c'est pour cela que j'ai clairement identifié (font rouge + commentaire) cet objet sur le diagramme d'objets. Cet objet hypothétique permet uniquement de pointer du doigt le fait que j'ai identifié un objet qui ne peut pas être représenté dans un diagramme d'objets découlant de notre diagramme de classes.

Je suppose que le Tonneau est un Obstacle infranchissable dans la **spécification globale** du jeu, puisqu'il apparaît la palette de conception de la Figure 2b de l'énoncé. Cependant, ça ne change rien au fait qu'on ne peut pas représenter cet objet dans notre diagramme d'objets.

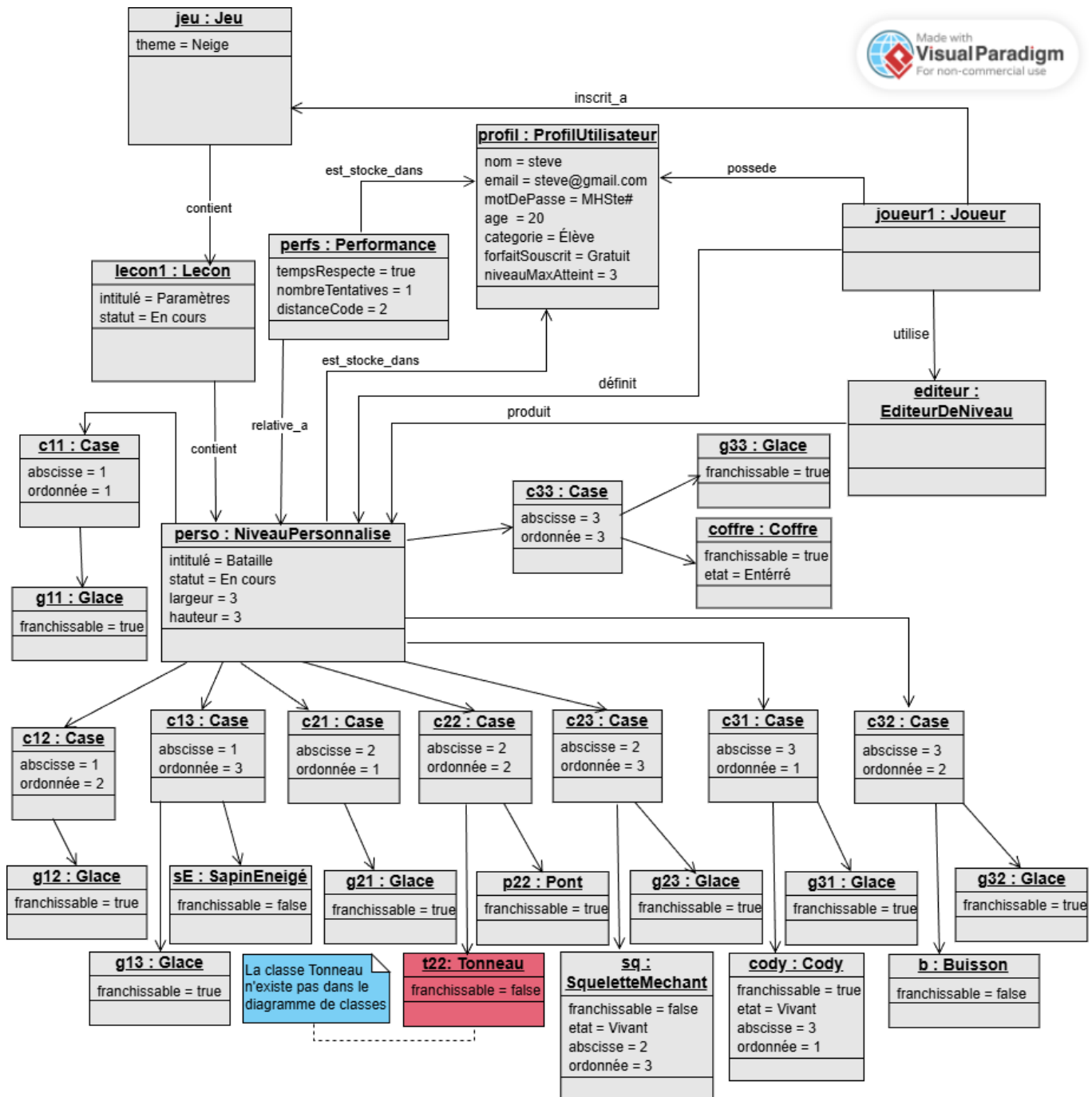


Figure 3: Diagramme d'Objets correspondant à la Figure fournie

2. Propriété du jeu qui n'est pas satisfaite par ce niveau :

Dans la question précédente, nous avons déjà clairement identifié un objet de type Tonneau qui ne devrait pas être là, puisque le Tonneau ne fait pas partie des types d'Obstacles dans la spécification et donc n'existe pas dans notre diagramme de classes.

Néanmoins, telle que cette question est posée, j'ai l'impression qu'il s'agit plutôt d'identifier une situation qui soit conforme à notre diagramme de classes, mais non conforme à la spécification. Il aurait pu s'agir de la présence d'un **Buisson** dans le niveau alors que le thème choisi est **Neige** (on le remarque au type des éléments de surface). Nous savons que le type d'obstacle

dépend du thème, c'est-à-dire que le seul type d'obstacle pouvant apparaître dans ce niveau est SapinEneigé.

Il s'agit là aussi d'une propriété qui n'est pas respectée par ce niveau, mais ça ne colle pas avec la question suivante puisque la contrainte OCL ci-dessous permet de vérifier cette propriété :

```
context Jeu
inv :
if self.thème = Theme::Plaine
then self.lecons.niveaus.cases.elementGraphiques->select(e | e.ocIsTypeOf(Obstacle))
->forAll(o | o.ocIsTypeOf(Buisson))
else if self.thème = Theme::Neige
then self.lecons.niveaus.cases.elementGraphiques->select(e | e.ocIsTypeOf(Obstacle))
->forAll(o | o.ocIsTypeOf(SapinEneigé))
else -- thème = Theme::Désert
self.lecons.niveaus.cases.elementGraphiques->select(e | e.ocIsTypeOf(Obstacle))
->forAll(o | o.ocIsTypeOf(PyloneRocheux))
endif
endif
```

Finalement, la propriété non respectée qui est attendue dans cette question est : "le placement des éléments graphiques doit laisser la possibilité d'atteindre le trésor".

- Non, il n'est pas possible de définir une contrainte OCL qui permette de vérifier cette propriété car OCL ne permet d'exprimer que des propriétés déclaratives, **statiques** et locales. Pour pouvoir définir une telle contrainte, il faudrait que le langage OCL permette de simuler le déroulement d'un niveau du jeu pour s'assurer que Cody atteigne le trésor, ce qui est impossible.

Question 4.5

Définissons un niveau original : voir Figure 4

Le thème choisi est **Plaine**.

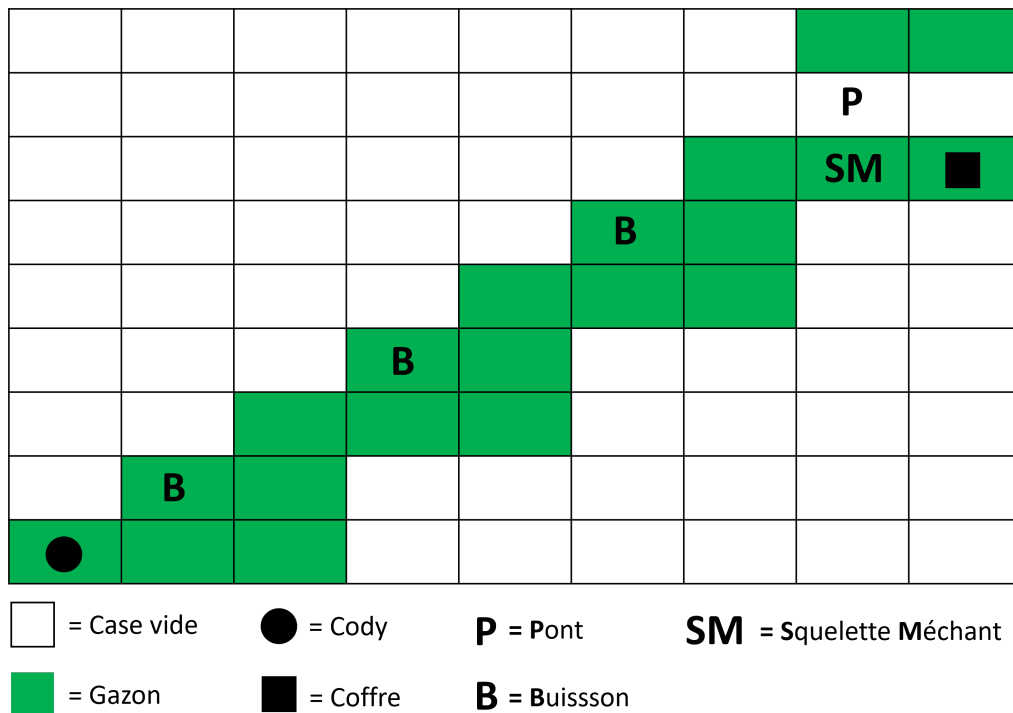


Figure 4: Niveau original

2 Modélisation de Play

Question 4.6

Première version d'un diagramme de classe UML qui fixe les éléments principaux : voir Figure 5

Puisqu'on ne modélise pas le détail du concept de Procédure, je n'ai pas représenté le fait qu'une procédure comporte un corps et des paramètres.

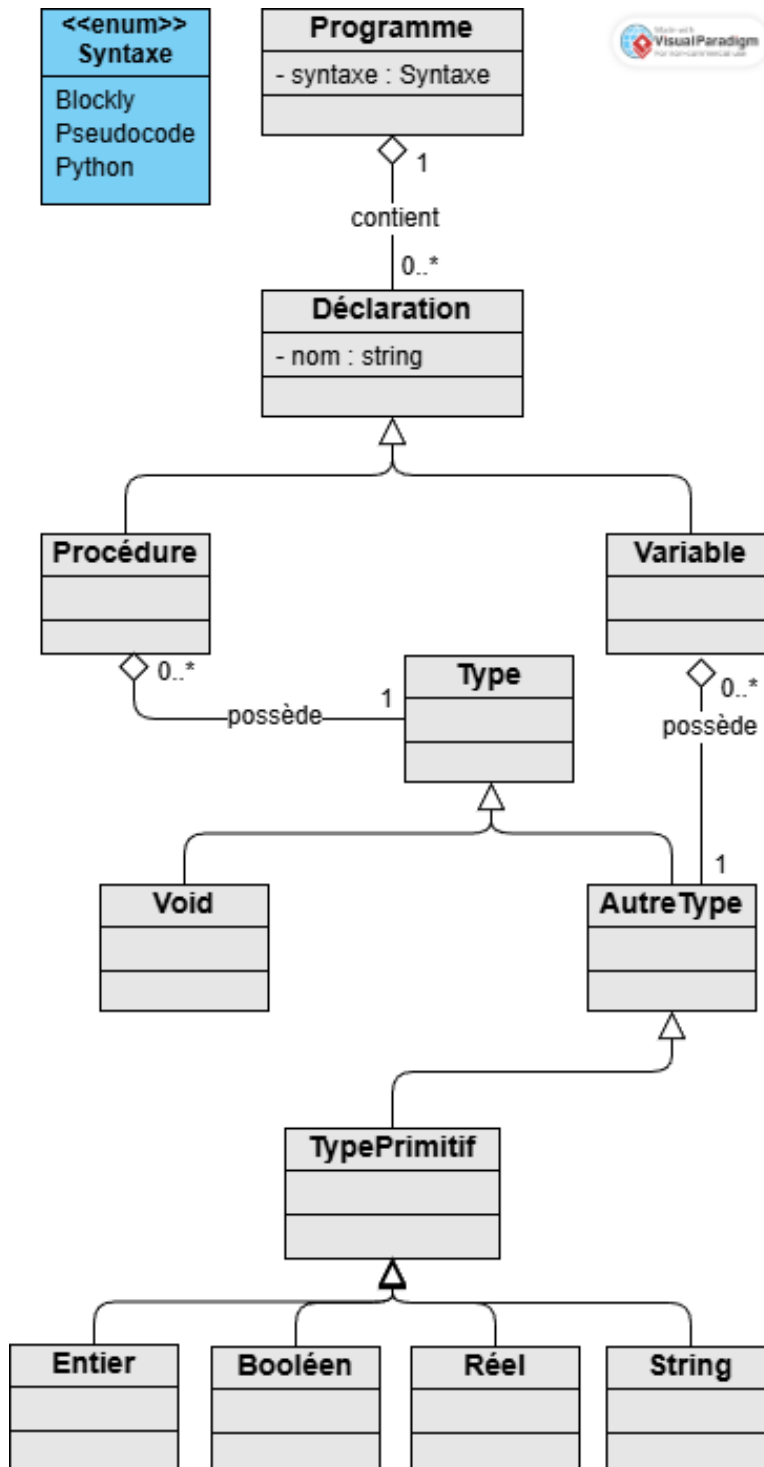


Figure 5: Première version d'un diagramme de classe UML de Play

Question 4.7

Raffinons la modélisation du concept de Déclaration : voir Figure 6

Hypothèses :

- Puisqu'on peut déclarer des types structurés dans un programme, alors on suppose qu'une déclaration peut être une déclaration de type structuré.
- On peut déclarer une variable (locale) au sein d'une procédure.
- On ne peut pas déclarer une procédure au sein d'une autre (pas d'imbrication de procédures).

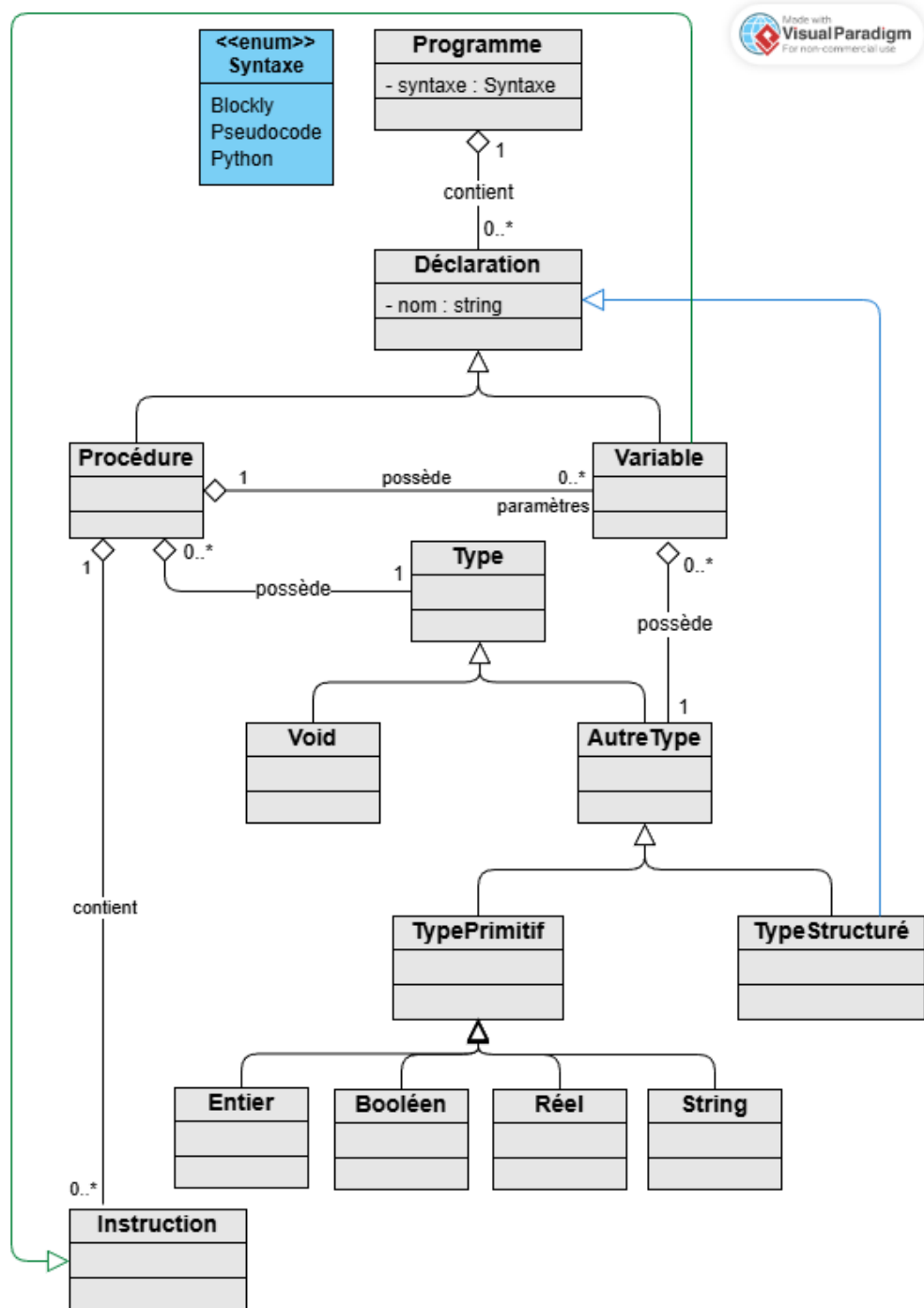


Figure 6: Diagramme de classe UML de Play + raffinement de Déclaration

Question 4.8

Raffinons le concept de Type en détaillant les types structurés : voir Figure 7

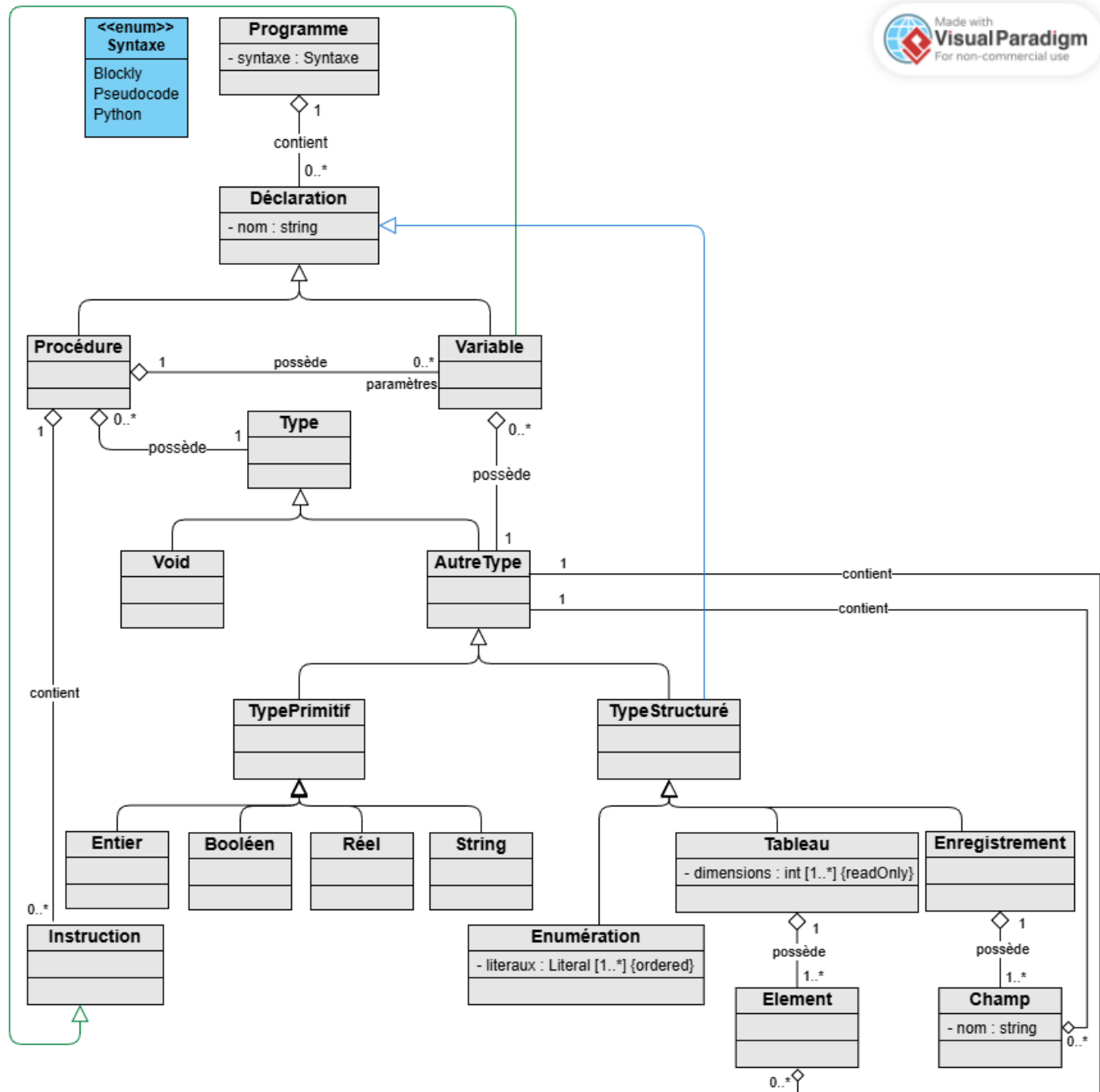


Figure 7: Diagramme de classe UML de Play + raffinement de TypeStructuré

Question 4.9

Diagramme d'Objets représentant tous les exemples de déclarations données en 3.2 : voir Figure 8

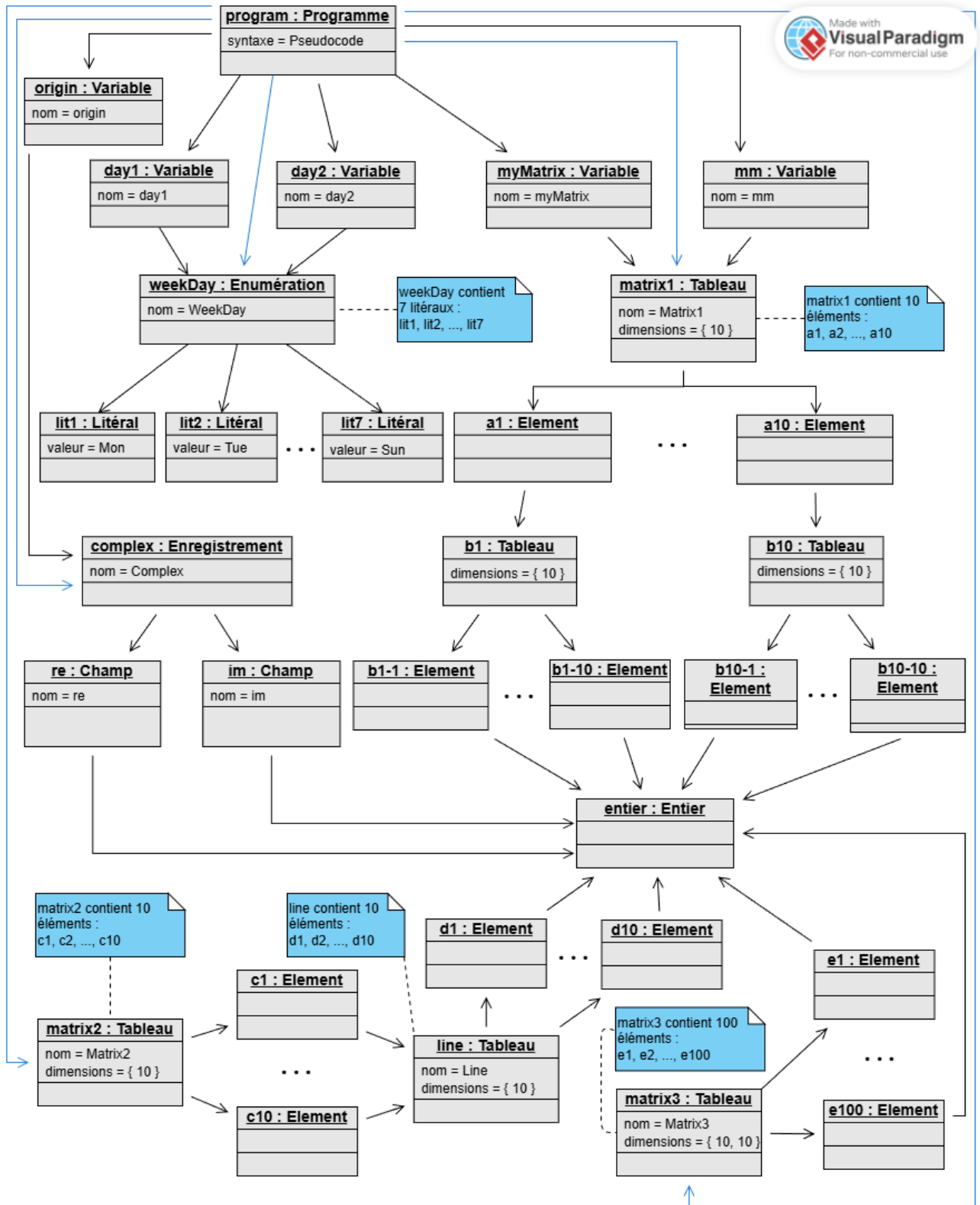


Figure 8: Diagramme d'Objet des exemples de déclarations

Question 4.10

Raffinons le détail du concept Instruction : voir Figure 9

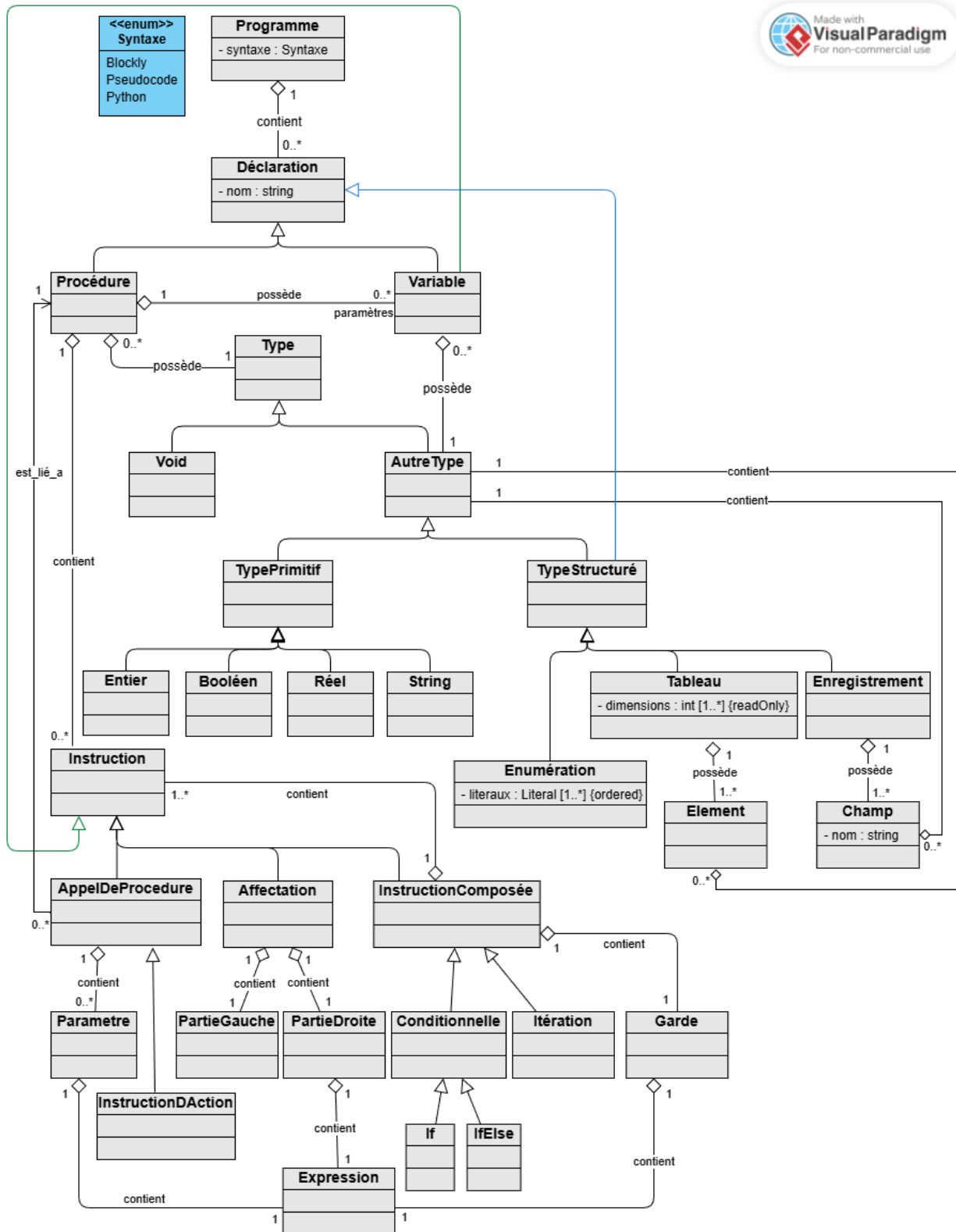


Figure 9: Diagramme de classe UML de Play + raffinement de Instruction

Question 4.11

Raffinons le concept Expression : voir Figure 10

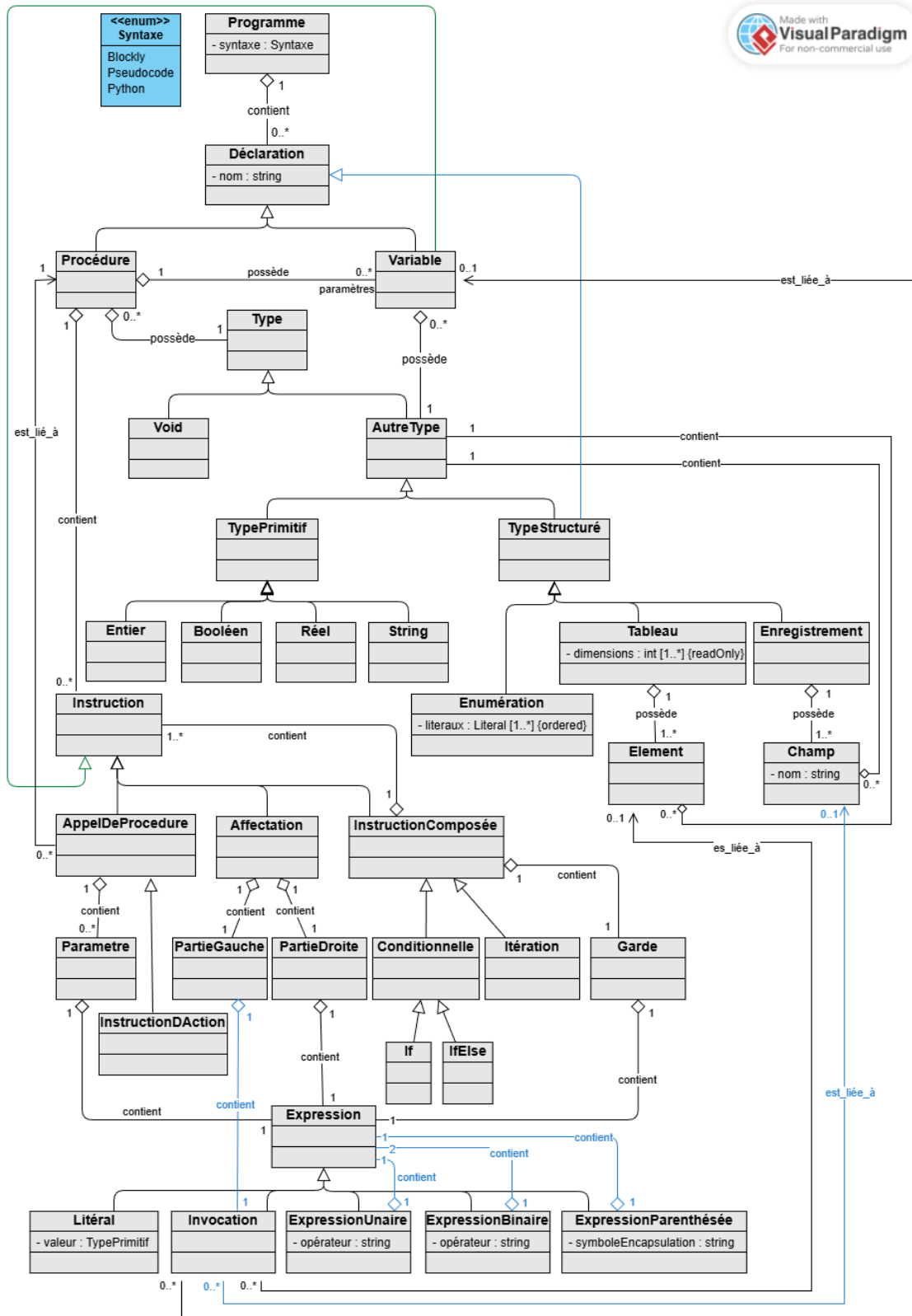


Figure 10: Diagramme de classe UML de Play + raffinement de Expression

Question 4.12

Spécifions une contrainte Ocl vérifiant l'unicité des déclarations au sein de leur contexte :

1. Les noms de procédures au sein d'une instance de programme :

```
context Programme
inv :
self.declarations->select(d | d.ocIsTypeOf(Procédure))
->forAll(p1, p2 | p1 <> p2 implies p1.nom <> p2.nom)
```
2. La procédure Cody au sein d'une instance de Program(me) :

```
context Programme
inv :
self.declarations->select(d | d.ocIsTypeOf(Procédure) and d.nom = Cody)
->size() = 1
```
3. Les noms de variables (globales) au sein d'une instance de programme :

```
context Programme
inv :
self.declarations->select(d | d.ocIsTypeOf(Variable))
->forAll(v1, v2 | v1 <> v2 implies v1.nom <> v2.nom)
```
4. Les noms des paramètres au sein d'une déclaration de procédure :

```
context Procédure
inv :
self.paramètres->forAll(p1, p2 | p1 <> p2 implies p1.nom <> p2.nom)
```
5. Les noms de variables au sein du corps d'une déclaration de procédure :

```
context Procédure
inv :
self.instructions->select(i | i.ocIsTypeOf(Variable))
->forAll(v1, v2 | v1 <> v2 implies v1.ocAsType(Variable).nom <> v2.ocAsType(Variable).nom)
```
6. Les noms des champs au sein d'une déclaration d'enregistrement :

```
context Enregistrement
inv :
self.champs->forAll(c1, c2 | c1 <> c2 implies c1.nom <> c2.nom)
```

Question 4.13

Spécifions une contrainte Ocl permettant de vérifier qu'une déclaration de type est bien formée :

1. la liste de champs d'un enregistrement est non-vide :
La cardinalité 1..* côté Champ de la l'association **Enregistrement** -> **Champ** assure déjà cette contrainte. Néanmoins, la contrainte OCL est la suivante :

```
context Enregistrement
inv :
self.champs->size() > 0
```
2. un tableau comporte au moins une dimension qui doit être strictement positive :

```
context Tableau
inv :
self.dimensions->select(d | d > 0)->size() > 0
```

Question 4.14

Spécifions en Ocl le contrat Ocl sur une opération Expression : : type() : Type qui renvoie le type d'une expression :

```
context Expression
def: type() : Type =
if self.oclIsTypeOf(Litéral) then self.oclAsType(Litéral).type()
else if self.oclIsTypeOf(ExpressionUnaire) then self.oclAsType(ExpressionUnaire).type()
else if self.oclIsTypeOf(ExpressionBinaire) then self.oclAsType(ExpressionBinaire).type()
else if self.oclIsTypeOf(ExpressionParenthésée) then self.oclAsType(ExpressionParenthésée).type()
else if self.oclIsTypeOf(Invocation) then self.oclAsType(Invocation).type()
else OclInvalid
endif endif endif endif endif

-- Litéral
context Litéral
def: type() : Type =
if self.oclIsTypeOf(Integer) then Entier
else if self.oclIsTypeOf(Real) then Réel
else if self.oclIsTypeOf(String) then String
else if self.oclIsTypeOf(Boolean) then Booléen
else OclInvalid
endif endif endif endif

-- Expression Unaire
context ExpressionUnaire
def: type() : Type =
if self.opérateur = "-"
and (self.sousExpression.type() = Entier or self.sousExpression.type() = Réel)
then self.sousExpression.type()
else if self.opérateur = "not" and self.sousExpression.type() = Booléen
then self.sousExpression.type()
else OclInvalid
endif endif

-- Expression Binaire
context ExpressionBinaire
def: type() : Type =
if (self.opérateur = "+" or self.opérateur = "-" or self.opérateur = "*"
or self.opérateur = "/")
and (self.sousExpressions->forall(se | se.type() = Entier or se.type() = Réel))
then (if self.sousExpressions->select(se | se.type() = Réel)->size() >= 1 then Réel else
Entier endif)
else if (self.opérateur = "or" or self.opérateur = "and" or self.opérateur = "xor")
and (self.sousExpressions->forall(se | se.type() = Booléen))
then Booléen
else if (self.opérateur = "<" or self.opérateur = "<=" or self.opérateur = ">"
or self.opérateur = ">=" or self.opérateur = "=" or self.opérateur = "<>")
and (self.sousExpressions->forall(se | se.type() = Entier or se.type() = Réel))
then Booléen
else OclInvalid
endif endif endif

-- Expression Parenthésée
context ExpressionParenthésée
def: type() : Type = self.sousExpression.type()
```

```
-- Invocation
context Invocation
def: type() : Type =
if self.variable <> null then self.variable.type
else if self.element <> null then self.element.type
else if self.champ <> null then self.champ.type
else OclInvalid
endif endif endif
```

Question 4.15

Spécifions le contrat Ocl sur une opération Instruction : : `estValide()` : boolean qui vérifie qu'une instruction est valide :

```
context Instruction
def: estValide() : Booléen =
if self.oclIsTypeOf(AppelDeProcédure) then self.oclAsType(AppelDeProcédure).estValide()
else if self.oclIsTypeOf(Affectation) then self.oclAsType(Affectation).estValide()
else if self.oclIsTypeOf(InstructionComposée) then self.oclAsType(InstructionComposée).estValide()
else false
endif endif endif

-- Appel de Procédure
context AppelDeProcédure
def: estValide() : Booléen =
if self.oclIsTypeOf(InstructionDAction) -- cas des procédures par défaut
then self.parametres->forall(p | p.expression.type() = Entier)
else ( -- cas des procédures déclarées dans le programme
self.procedure.type = Void -- le type de retour de toute procédure doit être Void
and
self.procedure.nom <> null -- réfère à une déclaration de procédure dans le programme
and
self.parametres->at(i).expression.type() = self.procedure.parametres->at(i).type
)
endif

-- Instruction Composée
context InstructionComposée
def: estValide() : Booléen = (self.garde.expression.type() = Booléen)

-- Affectation
context Affectation
def: estValide() : Booléen =
(self.partieGauche.invocation.type() = self.partieDroite.expression.type())
```

Question 4.16

En supposant l'existence d'une opération `prec_mouv()` : Déplacement qui retourne la direction du dernier déplacement effectué, spécifions les contrats Ocl sur l'ensemble des instructions d'actions primitives de déplacement (`up()`, `down()`, `right()`, `left()` et `jump()`) :

Gardons à l'esprit ici que l'axe des abscisses va de la gauche vers la droite et celle des ordonnées du haut vers le bas. De plus, l'abscisse d'une case est comprise entre 1 et largeur du niveau, tandis que l'ordonnée est comprise entre 1 et hauteur du niveau.

```

context Cody::up()
  post:
    if self.ordonnée@pre - 1 >= 1 then
      if self.case.niveau.cases->select(c | c.abscisse = self.abscisse@pre
        and c.ordonnée = self.ordonnée@pre - 1).elementGraphiques
        ->select(e | e.oclIsTypeOf(Obstacle))->notEmpty() then

        self.abscisse = self.abscisse@pre
        self.ordonnée = self.ordonnée@pre -- le déplacement n'est pas effectué
      else
        self.abscisse = self.abscisse@pre
        self.ordonnée = self.ordonnée@pre - 1
      endif
    else
      self.abscisse = self.abscisse@pre
      self.ordonnée = self.ordonnée@pre - 1
    endif

context Cody::down()
  post:
    if self.ordonnée@pre + 1 <= self.case.niveau.hauteur then
      if self.case.niveau.cases->select(c | c.abscisse = self.abscisse@pre
        and c.ordonnée = self.ordonnée@pre + 1).elementGraphiques
        ->select(e | e.oclIsTypeOf(Obstacle))->notEmpty() then

        self.abscisse = self.abscisse@pre
        self.ordonnée = self.ordonnée@pre -- le déplacement n'est pas effectué
      else
        self.abscisse = self.abscisse@pre
        self.ordonnée = self.ordonnée@pre + 1
      endif
    else
      self.abscisse = self.abscisse@pre
      self.ordonnée = self.ordonnée@pre + 1
    endif

context Cody::right()
  post:
    if self.abscisse@pre + 1 <= self.case.niveau.largeur then
      if self.case.niveau.cases->select(c | c.abscisse = self.abscisse@pre + 1
        and c.ordonnée = self.ordonnée@pre).elementGraphiques
        ->select(e | e.oclIsTypeOf(Obstacle))->notEmpty() then

        self.abscisse = self.abscisse@pre -- le déplacement n'est pas effectué
        self.ordonnée = self.ordonnée@pre
      else
        self.abscisse = self.abscisse@pre + 1
        self.ordonnée = self.ordonnée@pre
      endif
    else
      self.abscisse = self.abscisse@pre + 1
      self.ordonnée = self.ordonnée@pre
    endif

```

```

context Cody::left()
  post:
    if self.abscisse@pre - 1 >= 1 then
      if self.case.niveau.cases->select(c | c.abscisse = self.abscisse@pre - 1
        and c.ordonnée = self.ordonnée@pre).elementGraphiques
        ->select(e | e.oclIsTypeOf(Obstacle))->notEmpty() then

        self.abscisse = self.abscisse@pre -- le déplacement n'est pas effectué
        self.ordonnée = self.ordonnée@pre
      else
        self.abscisse = self.abscisse@pre - 1
        self.ordonnée = self.ordonnée@pre
      endif
    else
      self.abscisse = self.abscisse@pre - 1
      self.ordonnée = self.ordonnée@pre
    endif
context Cody::jump()
  post:
    if self.prec_mouv() = null then -- on ne peut pas sauter sans élan
      self.abscisse = self.abscisse@pre
      self.ordonnée = self.ordonnée@pre
    else
      if self.prec_mouv() = "up" then
        -- similaire à Cody::up() mais avec un déplacement potentiel de 2 vers le haut
        if self.ordonnée@pre - 2 >= 1 then
          if self.case.niveau.cases->select(c | c.abscisse = self.abscisse@pre
            and c.ordonnée = self.ordonnée@pre - 2).elementGraphiques
            ->select(e | e.oclIsTypeOf(Obstacle))->notEmpty() then

            self.abscisse = self.abscisse@pre
            self.ordonnée = self.ordonnée@pre -- le déplacement n'est pas effectué
          else
            self.abscisse = self.abscisse@pre
            self.ordonnée = self.ordonnée@pre - 2
          endif
        else
          self.abscisse = self.abscisse@pre
          self.ordonnée = self.ordonnée@pre - 2
        endif
      else
        if self.prec_mouv() = "down" then
          -- similaire à Cody::down() mais avec un déplacement potentiel de 2 vers le bas
        else
          if self.prec_mouv() = "right" then
            -- similaire à Cody::right() mais avec un déplacement potentiel de 2 vers
            la droite
          else
            if self.prec_mouv() = "left" then
              -- similaire à Cody::left() mais avec un déplacement potentiel de 2
              vers la gauche
            else
              self.abscisse = self.abscisse@pre
              self.ordonnée = self.ordonnée@pre
            endif
          endif
        endif
      endif
    endif
  endif
end if
end if
end if
end if

```

Question 4.17

Pour la Figure 3 fournie,

1. Donnons le code Play permettant de terminer le niveau, lorsque le sapin a été retiré du niveau :

Puisque Cody ne peut pas sauter sans élan, alors on ne peut pas juste sauter par dessus le buisson pour atteindre le trésor plus rapidement, il faut nécessairement passer par la case où se trouvait le sapin. Le code Play correspondant est le suivant :

```
procedure Cody() {  
  up(2)  
  right(2)  
  fight()  
  down(2)  
  dig()  
}
```

2. Donnons le Diagramme d'Objets Uml correspondant à ce code : voir Figure 11

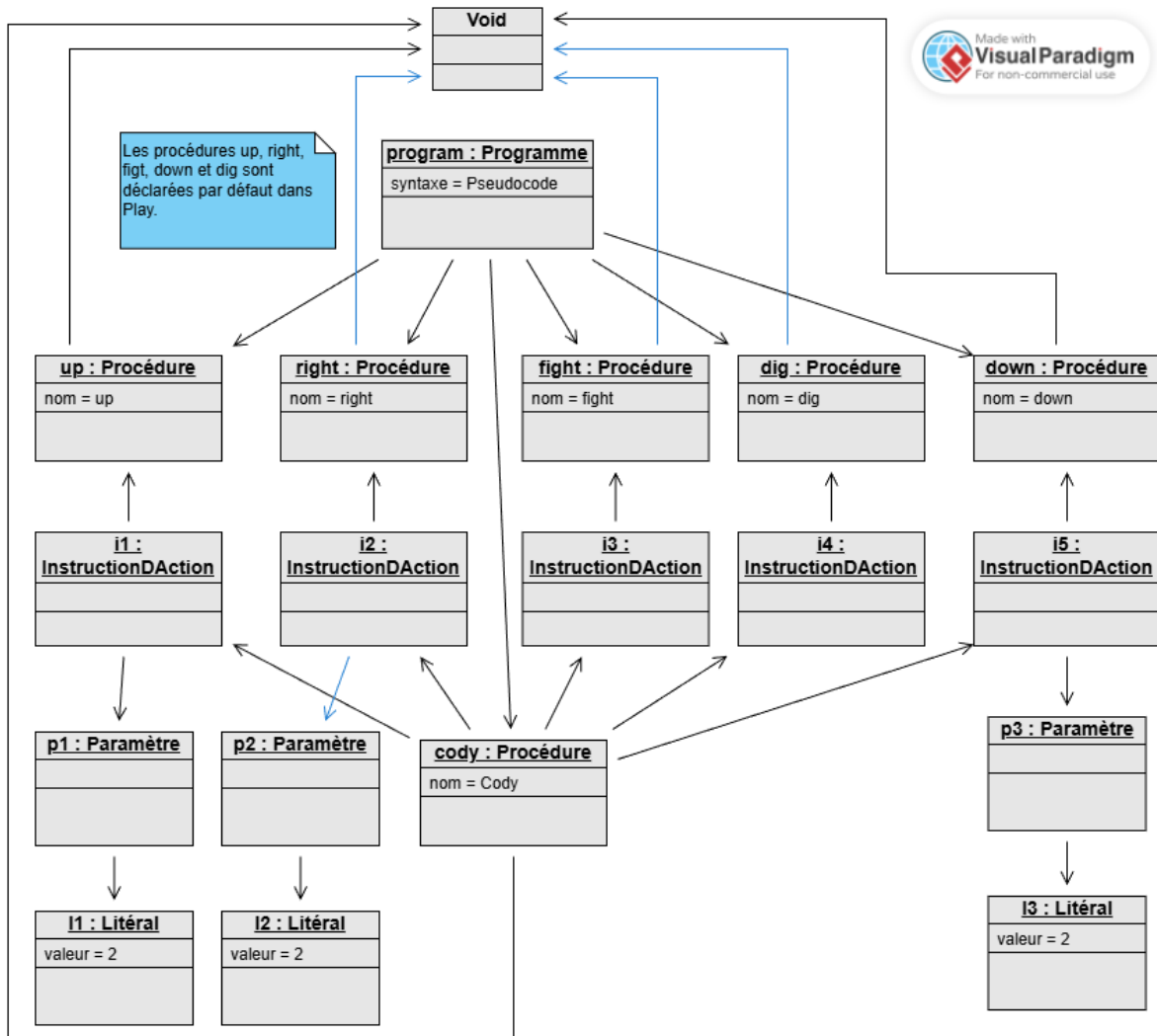


Figure 11: Diagramme d'Objet correspondant au code du niveau fourni

Question 4.18

Donnons le code Play permettant de résoudre notre niveau original défini dans la Question 4.5 :

Il est question ici de donner un code Play permettant d'illustrer les concepts suivants :

- boucles imbriquées : la procédure Cody contient une boucle simple et appelle avoidObstacle qui appelle à son tour manyRights qui contient aussi une boucle simple, d'où l'imbrication de boucles.
- portée des variables : deux variables locales de même nom `i` sont définies dans deux procédures différentes (`manyRights` et `Cody`). Chacune d'elles a son propre contexte, ce qui représente bien le concept de portée des variables.
- récursivité : la procédure `manyUps` est récursive.

La procédure `manyRights` permet d'aller une ou plusieurs fois à droite, `manyUps` d'aller une ou plusieurs fois en haut et `avoidObstacle` d'éviter un obstacle.

```
procedure manyRights(n : integer) {  
  var i : integer;  
  i := 0;  
  while i < n {  
    right();  
    i := i + 1;  
  }  
}
```

```
procedure manyUps(n : integer) {  
  if(n > 0){  
    up();  
    manyUps(n-1);  
  }  
}
```

```
procedure avoidObstacle() {  
  manyRights(2);  
  manyUps(2);  
}
```

```
procedure Cody() {  
  var i : integer;  
  i := 3;  
  
  while i > 0 {  
    avoidObstacle();  
    i := i - 1;  
  }  
  
  fight();  
  right(2);  
  dig();  
}
```